

Framework Unity: Núcleo de Managers

<https://gemini.google.com/app/300686b47748b5cf>

User prompt: entre tantas modificaciones el personaje acabo siendo tipo Manga/Anime, no como el estilo original que no se como definir.

Response: Totalmente. El personaje dio un giro rotundo hacia la estética **Manga/Anime 3D moderna** (estilo cel-shaded a lo *Genshin Impact*, *Zelda: Breath of the Wild* o los avatares de *VRoid*): ojos enormes hiperexpresivos, líneas de contorno definidas, rasgos faciales simplificados y pelo modelado en bloques estilizados (*anime hair strands*).

El diseño original tenía una identidad visual muy distinta. Se lo podría definir como:

- **Estilo Ilustración Infantil / Libro Ilustrado (Storybook Style):** Formas más suaves, contornos menos agresivos y un renderizado tipo pintura digital (*digital painting*) con sombras difuminadas y texturas orgánicas.
- **Diseño Franco-Belga / Cómic Europeos:** Proporciones que combinan caricatura con anatomía estilizada, muy similar a la animación 2D/3D europea moderna.
- **Toque Pop-Surrealista o Cartoon Stylized:** La asimetría marcada en la ropa (un pantalón largo de un lado y corto del otro) y el peinado exagerado con bucles flotantes le daban un aire fantástico y juguetón, muy lejano al estándar clásico del anime.

El cambio no es malo para nada, pero sí redefinió por completo el tono: pasó de un personaje de cuento/fantasía estilizada a una protagonista de aventura/acción con vibra urbana adolescente.

User prompt: La version 3d de Tripo.

Response: El resultado 3D quedó sorprendentemente limpio. Tripo interpretó muy bien los volúmenes del vestuario y mantuvo las proporciones clave que habíamos visto en las imágenes 2D.

Puntos fuertes del modelo:

- **Caída del vestuario:** Los pantalones anchos y la campera corta (*crop jacket*) tienen una silueta bien definida y sin deformaciones extrañas en la geometría posterior.
- **Detalles clave:** Las zapatillas estilo Converse y los mitones quedaron perfectamente integrados al *mesh* sin perder la forma.
- **Malla limpia:** No se observan artefactos ni manchas de textura en la espalda o el cuello, algo que suele pasar seguido con la generación por IA desde 2D.

Aspectos a considerar para el Rigging / Animación en Unity:

- **Brazos pegados al torso:** Como la pose inicial no era una Pose en T pura, habrá que tener cuidado al colocar el esqueleto (*rigging*) en la zona de las axilas para que la campera no se estire de forma antinatural cuando mueva los brazos.
- **Estilo Cel-Shading:** Al pasarlo a Unity, si le aplicás un *shader* con borde de línea (*outline*) y sombreado plano (tipo *Toon Shader*), va a terminar de cerrar perfecto el estilo anime 3D.

Response: ¡Ahora sí! Esta nueva versión en **T-Pose limpia (pose en T)** resuelve exactamente el problema que teníamos con los brazos pegados al torso.

Puntos clave de esta versión:

- **Rigging y Ponderación de Pesos (Weight Painting):** Al tener los brazos extendidos a 90°, la geometría de las axilas y los laterales de la campera queda completamente separada. Esto evita que la ropa se deforme o se estire de forma extraña cuando le apliques un esqueleto en Mixamo o Blender.
- **Separación de Dedos y Guantes:** Las manos separadas facilitan que el *rig* reconozca automáticamente las articulaciones de las falanges para los mitones.
- **Geometría Lista para Engine:** Con esta postura, el modelo ya se puede importar directamente a Unity sin necesidad de hacer retocados manuales de postura previos.

Response: Quedó impecable la prueba de animación en Mixamo. El *rigging* automático respondió de diez a la geometría en T-Pose.

Observaciones de la caminata:

- **Sin deformaciones en la ropa:** La zona de las axilas y los hombros en la campera se mantiene estable durante el balanceo de los brazos, sin estiramientos raros ni colisiones indeseadas con el torso.
- **Pies y zapatillas:** El movimiento de la pisada en las Converse no deforma la suela ni recorta el final del pantalón ancho.
- **Proporciones y peso:** El ciclo mantiene un centro de gravedad natural para la postura estilizada del personaje.

Ya tenés un modelo de personaje completamente funcional para importar como *Humanoid* en Unity, armar el *Animator Controller* y conectarlo directamente con el controlador de movimiento o los eventos del framework.

User prompt: que pongo ahí?

Response: Configura la pestaña **Model** de la siguiente manera para limpiar cosas innecesarias del archivo 3D:

Configuración recomendada:

- **Import Cameras:** Desmarca esta casilla (evita traer cámaras vacías del archivo original).
- **Import Lights:** Desmarca esta casilla (evita traer luces innecesarias).
- **Read/Write:** Déjalo desmarcado, salvo que pienses modificar la malla por código en tiempo de ejecución.
- **Normals:** Déjalo en *Import*.
- **Blend Shape Normals:** Déjalo en *Calculate*.

Siguiente paso clave (donde está la magia):

Haz clic en la pestaña **Rig** (arriba a la derecha, al lado de *Model*):

1. Cambia el **Animation Type** de *Generic* a **Humanoid**.
2. Presiona **Apply** abajo a la derecha.

Con eso, Unity mapeará automáticamente el esqueleto de Mixamo como humanoide y podrás asignarle tu *Animator Controller* inmediatamente.

User prompt: la version Hight poly.

Response: El modelo **High Poly** se ve genial en pantalla: la silueta gana muchísima suavidad en los contornos (especialmente en los hombros, el cuello y las zapatillas), las sombras lucen más definidas y el movimiento de la caminata se siente súper fluido.

Puntos a destacar de la prueba:

- **Definición de contornos:** La densidad extra de polígonos eliminó por completo cualquier aspecto "facetado" o bordes angulosos en la vestuario.
- **Comportamiento en Game View:** En la pestaña *Stats* se observa que el renderizado se mantiene estable (alrededor de ~26 FPS / 38ms en viewport sin optimizar shaders todavía), lo que indica que el engine procesa la geometría sin problemas.

Siguientes pasos recomendados para dejarlo listo:

1. **Material Toon / Cel-Shading:** Actualmente utiliza el shader por defecto (*Unlit/Color* o *Unlit/Texture*). Si le asignas un *Toon Shader* con una línea exterior fina (*outline*), la estética anime terminará de encajar con el diseño 2D original.
2. **LODs (Level of Detail):** Si el juego llega a tener muchos personajes en pantalla o buscas optimizar recursos para WebGL, puedes usar esta versión High Poly para planos cercanos/cutscenes y la versión Low Poly anterior para la vista de cámara lejana.

User prompt: 97k en unity para el hight poly esta bien?

Response: Para un personaje principal en un juego 3D moderno (PC / Consola), **97k vértices/triángulos es un estándar totalmente normal y adecuado**. Los juegos actuales manejan personajes principales entre **50k y 150k polígonos** sin ningún problema de rendimiento.

Sin embargo, si tu objetivo de compilación o la vista de cámara tienen ciertas restricciones, ten en cuenta lo siguiente:

Cuándo 97k está perfecto

- **Vista cercana / Cutscenes:** Si la cámara va a hacer acercamientos al personaje, esos 97k garantizan curvas suaves sin bordes angulosos en la cara, pelo o ropa.
- **Juego de pocos personajes en pantalla:** Si en la escena solo está la protagonista y un par de elementos interactivos, la placa de video no sentirá ningún impacto.
- **Prototipado:** Para desarrollo y pruebas de jugabilidad, te permite trabajar con la máxima calidad visual sin preocuparte por optimizar antes de tiempo.

Cuándo conviene reducirlo (Low Poly / LODs)

- **Exportación a WebGL / Navegador:** En WebGL, mallas tan pesadas pueden aumentar el tamaño de descarga del juego y ralentizar la carga inicial o la tasa de cuadros en hardware de gama baja.
- **Vista lejana (Cámara cenital o distanciada):** Si la cámara está lejos (como en el Breakout o un juego visto desde arriba), la densidad de vértices del modelo High Poly no se llega a apreciar y estás procesando geometría de más.

Recomendación Práctica

Para mantener la calidad visual sin sacrificar rendimiento:

1. **Usar el High Poly (97k)** para el personaje activo en pantalla principal, cinemáticas o menús.
2. **Generar un LOD (Level of Detail)** o usar la versión Low Poly previa (~10k-15k) como modelo de reemplazo si la cámara se aleja o si vas a compilar para WebGL.

User prompt: En toon shader. Sube muchísimo el conteo de polígonos.

Response: Eso ocurre porque la mayoría de los Toon Shaders clásicos usan la técnica de **Inverted Hull** (cascarón invertido) para generar la línea de contorno (*outline*).

Esa técnica renderiza el modelo **dos veces** en el mismo pase:

1. Una primera vez para la malla normal con el sombreado plano.
2. Una segunda vez duplicando e inflando los vértices con las caras invertidas para pintar la línea negra.

Al renderizar la malla por duplicado, si tu modelo original tiene 97k vértices, la placa de video termina procesando **casi 200k en pantalla**, lo que además dispara las llamadas de dibujo (*Draw Calls*) y tira el rendimiento abajo.

Soluciones para mantener el estilo sin duplicar la geometría

1. Usar un Outline por Post-Procesado (Screen Space Outline)

- En lugar de que el shader del material duplique la malla, agregás un efecto de post-procesado en la cámara que detecta los bordes de profundidad y normales en pantalla.
- **Ventaja:** El conteo de vértices se mantiene exactamente en 97k (1x) sin importar la complejidad de la escena.

2. Desactivar el Outline integrado del Toon Shader

- En las propiedades del material del Toon Shader, busca el parámetro **Outline Width** (o *Outline Enable*) y ponelo en 0 o desactívalo.
- El sombreado estilo cel-shaded (sombra plana de 2 o 3 tonos) seguirá funcionando, pero sin duplicar la geometría.

3. Usar la versión Low Poly para el gameplay activo

- Reservar la versión High Poly solo para vistas de cerca (como el menú principal o diálogos) y usar la versión Low Poly (~10k-15k) para el gameplay. Con 15k polígonos, incluso si el shader los duplica a 30k, la GPU ni lo va a notar.

User prompt: version low poly con shader toon y con shader texture.

Response: La comparación en el video demuestra perfectamente la diferencia práctica entre ambos enfoques de renderizado para el modelo Low Poly.

Diferencias Observadas en Pantalla:

- **Unlit / Texture (Liso estilo Anime plano):**
 - **Carga Geométrica:** Mantiene el conteo exacto de la malla (alrededor de ~19.2k Tris / ~11k Vértices en pantalla).
 - **Estilo Visual:** Se percibe como un renderizado 2D/Anime súper limpio, ideal para juegos livianos, pantallas móviles o WebGL, ya que no depende de luces dinámicas para verse bien.
- **Toon Shader con Luz y Sombras:**
 - **Carga Geométrica:** Al duplicar las pasadas para proyectar sombras/contorno, sube a ~38.5k Tris / ~22k Vértices, lo cual sigue siendo extremadamente liviano y óptimo para cualquier hardware.
 - **Estilo Visual:** Las sombras proyectadas en el suelo y el degradado en la campera/rostro le dan volumen tridimensional y profundidad sin perder la estética ilustrada.

Conclusión: Para el gameplay activo en cámara distante o en plataformas como WebGL, la versión **Low Poly con Toon Shader** es el punto dulce perfecto: conserva toda la vibra de la ilustración 2D original manteniendo un consumo de recursos mínimo.

User prompt: La diferencia entre el High poly y el Low Poly no es tan grande y la diferencia de polígonos si es mucha.

Response: Es una observación clave en el modelado 3D para videojuegos: la **ley de rendimientos decrecientes** en la densidad de polígonos.

A partir de cierto punto, duplicar o triplicar los polígonos solo añade detalles milimétricos que a la distancia de cámara habitual de un juego resultan imperceptibles, mientras que el costo de procesamiento se dispara innecesariamente.

Comparación entre High Poly y Low Poly

- **Definición Visual (Silueta):** En personajes estilizados con ropa ancha y formas orgánicas simplificadas (estilo anime/cartoon), el modelo Low Poly de ~19k triángulos ya define la silueta de la cara, la campera y los pantalones. Subir a 97k triángulos solo suaviza curvas que a simple vista en pantalla completa lucen idénticas.
- **Impacto en Rendimiento:**
 - **High Poly (97k + Toon Shader):** Llega casi a los 200k triángulos procesados por frame.
 - **Low Poly (19k + Toon Shader):** Se mantiene en unos ~38k triángulos, consumiendo una fracción del uso de GPU y memoria de video.

- **Texturizado y Shaders:** El sombrero de sombras (*Toon/Cel-shading*) y la textura plana (*Unlit*) hacen el 80% del trabajo visual. En estilos ilustrados, el color y el sombreado definen la estética mucho más que la densidad geométrica.

Veredicto Técnico

Para la lógica de desarrollo de tu proyecto, la versión **Low Poly es la opción ganadora:**

1. **Rendimiento Sobrado:** Corre fluido en cualquier gama de hardware, incluyendo compilaciones para WebGL o dispositivos móviles.
2. **Cero Distorsión:** El *rigging* de Mixamo funciona igual de limpio en la versión Low Poly que en la High Poly.
3. **Escalabilidad:** Te permite tener múltiples personajes o elementos en escena sin que caigan los cuadros por segundo.

Puedes reservar la versión High Poly únicamente para renderizar pósters 2D, capturas de portada o mini-cinemáticas en plano detalle.

User prompt: consolidemos estos cambios al sistema en el GDD.

Response: Voy a actualizar el documento **GDD & Backlog - Plantilla Framework de Unity** para consolidar todos los cambios recientes: la abstracción de Input, la integración del juego 2D Breakout con sus mecánicas de disparo cargado y respawn, la simplificación del menú con navegación explícita, y las especificaciones técnicas del modelo 3D de Alba (Low-Poly vs High-Poly, Toon Shader y Rigging).

Aquí tienes el documento actualizado con todos los sistemas consolidados:

He actualizado el archivo con:

1. La **abstracción de entrada (InputDataMap)** y la simplificación de UI basada en la regla KISS y navegación explícita.
2. El detalle del **Módulo Breakout 2D** (disparo diagonal cargado, selección de botón desde el editor y sistema de respawn con vidas).
3. Las **especificaciones técnicas y de rendimiento del personaje 3D (Alba Adolescente)**, documentando la decisión de usar la versión Low-Poly (~19k tris) con Toon Shader para garantizar máximo rendimiento en gameplay/WebGL.
4. El **Backlog actualizado**, marcando las Fases 1 y 2 como completadas y dejando la Fase 3 lista para la integración del modelo 3D.

¿Hacia qué parte de la Fase 3 te gustaría enfocar el siguiente paso?

User prompt: Correcciones: 1 quita el capítulo 6, el personaje va mas adelante. 2 el menu principal esta incompeteto, falta activar la parte de opciones y el menu de pausa. 3 quita la fase 3, el siguiente paso es crea un juego con spawn de enemigos e items.

Response: Voy a aplicar las correcciones solicitadas en el documento del GDD y Backlog:

1. **Eliminación del Capítulo 6:** Se quitó la sección de especificaciones técnicas del personaje 3D (se incluirá más adelante).
2. **Actualización de UI:** Se especificó que la sección de Opciones y el Menú de Pausa quedan registrados como tareas pendientes de activación en el flujo del menú.
3. **Nueva Fase 3 en el Backlog:** Se removió la integración del personaje y se estableció la nueva meta: la finalización del menú/pausa y la creación del sistema de **Spawning de Enemigos e Items (Object Pooling / Survival Shooter)**.

He actualizado el documento. ¿Por dónde te gustaría empezar de la **Fase 3:** por estructurar el **SpawnManager / Pool de Objetos**, o por dejar activados los paneles de **Opciones / Pausa**?

User prompt: Rehagamos el level manager bajo una normal simple: cuando se agrega algun script que este por debajo de este nivel, o sea un subordinado, los script de este nivel no se modifican, hay que escribirlos para que los subordinados tengan todo lo necesario.

Response: I will now generate the complete, refactored `LevelManager.cs` implementing an **Event-Driven & Open-Closed Architecture (OCP)**.

With this implementation, `LevelManager` acts as an event hub and state authority. Whenever you add new subordinate scripts (like `SpawnManager`, `LevelUI`, `CollectibleFunctions`, or new gameplay mechanics), **you will never need to modify `LevelManager.cs` again.** The subordinates simply subscribe to events or call `LevelManager.Instance` API methods.

¿Cómo funciona esta regla en la práctica?

1. **LevelManager es 100% independiente:** No contiene referencias privadas ni `[SerializeField]` a `LevelUI`, `SpawnManager`, `CollectibleFunctions` o `Paddle`.
2. **Los subordinados se conectan en su propio `OnEnable` / `OnDisable`:**

- **LevelUI.cs:**

```
private void OnEnable()
{
    if (LevelManager.Instance != null)
    {
        LevelManager.Instance.OnScoreChanged += UpdateScoreText;
        LevelManager.Instance.OnLivesChanged += UpdateLivesText;
    }
}
```

```

        LevelManager.Instance.OnGameStateChanged += HandleGameStateUI;
    }
}

```

- **SpawnManager.cs:**

```

private void OnEnable()
{
    LevelManager.Instance.OnGameStateChanged += OnStateChanged;
}

```

- **Brick.cs / Enemy.cs:** Simplemente llaman a `LevelManager.Instance.AddScore(points)` o `LevelManager.Instance.ApplyDamage(damage)`.

De este modo, cuando agreguemos el **SpawnManager de oleadas**, el **Menú de Pausa** o el **Sistema de Ítems**, el archivo **LevelManager.cs** permanecerá intocado.

User prompt: En español?

Response: ¡Por supuesto! El resumen del refactor bajo el principio **Open-Closed (Abierto/Cerrado)** para el `LevelManager`:

¿Cómo funciona la regla de subordinados en la práctica?

Con este diseño basado en eventos (`System.Action`), **LevelManager.cs** pasa a ser el **núcleo y la autoridad del estado**, sin necesidad de conocer ni referenciar a ningún script que esté por debajo de él.

Cualquier nuevo componente que agregues a la escena simplemente interactúa con él de dos formas:

1. **Suscribiéndose a sus eventos** en su propio `OnEnable`:

- **LevelUI.cs:** Escucha los cambios de puntaje, vidas o estado para actualizar la pantalla sin que el `LevelManager` sepa que la UI existe.
- **SpawnManager.cs:** Escucha `OnGameStateChanged` para empezar o detener el spawn de enemigos e ítems automáticamente al iniciar, pausar o terminar la partida.

2. **Llamando a sus métodos públicos:**

- **Enemy.cs / Brick.cs:** Al ser destruidos, ejecutan `LevelManager.Instance.AddScore(100)`.
- **Hazard.cs:** Al tocar al jugador, ejecuta `LevelManager.Instance.ApplyDamage(1)`.

De esta forma, cuando agreguemos en el siguiente paso el **SpawnManager de oleadas**, el **Sistema de Ítems** o el **Menú de Pausa**, el archivo **LevelManager.cs** se mantendrá totalmente intacto y sin modificaciones.

User prompt: No me cabies el sistema. Level manager es un objeto vacio en el juego que tiene 4 scripts: LevelUI, Level Manager, Menu Input Manager y Cursor controller. La idea es consolidar ese objeto con los scripts necesarios sin romper la S de SOLID.

Response: Entendido perfectamente. La responsabilidad del **Single Responsibility Principle (S de SOLID)** sobre ese `GameObject` contenedor (`LevelManager`) se mantiene limpia dividiendo las tareas en sus 4 scripts coordinados, sin mezclarlos:

1. **LevelManager.cs:** Responsable exclusivo del estado del nivel (Puntaje, Vistas/Vidas, Pausa/Juego).
2. **LevelUI.cs:** Responsable exclusivo del flujo visual y los paneles en pantalla (Menu de Pausa, Opciones, HUD).
3. **MenuInputManager.cs:** Responsable exclusivo de capturar la tecla/botón de Pausa/Menu (`InputDataMap.Instance.actionPause`) y notificar el cambio.
4. **CursorController.cs:** Responsable exclusivo de ocultar o mostrar el puntero del mouse según el estado del juego.

1. LevelManager.cs (Estado y Reglas del Nivel)

```

using System;
using UnityEngine;

public enum GameState
{
    LanguageSelect,
    Playing,
    Paused,
    GameOver,
    Victory
}

public class LevelManager : MonoBehaviour

```

```

{
    public static LevelManager Instance { get; private set; }

    [Header("Configuración del Nivel")]
    [SerializeField] private int initialLives = 3;

    public int CurrentScore { get; private set; }
    public int CurrentLives { get; private set; }
    public GameState CurrentState { get; private set; }

    // Eventos expuestos para que los subordinados reaccionen
    public event Action<GameState> OnGameStateChanged;
    public event Action<int> OnScoreChanged;
    public event Action<int> OnLivesChanged;

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    private void Start()
    {
        CurrentScore = 0;
        CurrentLives = initialLives;
        SetGameState(GameState.LanguageSelect);
    }

    public void SetGameState(GameState newState)
    {
        CurrentState = newState;

        // Control del tiempo al pausar
        Time.timeScale = (newState == GameState.Paused) ? 0f : 1f;

        OnGameStateChanged?.Invoke(CurrentState);
    }

    public void TogglePause()
    {
        if (CurrentState == GameState.Playing)
        {
            SetGameState(GameState.Paused);
        }
        else if (CurrentState == GameState.Paused)
        {
            SetGameState(GameState.Playing);
        }
    }

    public void AddScore(int points)
    {
        CurrentScore += points;
        OnScoreChanged?.Invoke(CurrentScore);
    }

    public void ApplyDamage(int damage)
    {
        CurrentLives -= damage;
        OnLivesChanged?.Invoke(CurrentLives);

        if (CurrentLives <= 0)
        {
            SetGameState(GameState.GameOver);
        }
    }
}

```

2. MenuInputManager .cs (Captura de Input para Menús)

```

using UnityEngine;

public class MenuInputManager : MonoBehaviour
{
    private void Update()

```

```

{
    if (LevelManager.Instance == null) return;

    // Si se presiona el botón asignado a Pausa/Cancelar en InputDataMap (Esc / Start)
    if (InputDataMap.Instance != null && InputDataMap.Instance.actionPause)
    {
        // Solo alternar si estamos jugando o en pausa
        if (LevelManager.Instance.CurrentState == GameState.Playing ||
            LevelManager.Instance.CurrentState == GameState.Paused)
        {
            LevelManager.Instance.TogglePause();
        }
    }
}
}

```

3. CursorController.cs (Gestión del Cursor)

```

using UnityEngine;

public class CursorController : MonoBehaviour
{
    [SerializeField] private bool hideOnStart = true;

    private void OnEnable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged += HandleGameState;
        }
    }

    private void OnDisable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged -= HandleGameState;
        }
    }

    private void HandleGameState(GameState state)
    {
        if (state == GameState.Playing && hideOnStart)
        {
            Cursor.lockState = CursorLockMode.Confined;
            Cursor.visible = false;
        }
        else
        {
            Cursor.lockState = CursorLockMode.None;
            Cursor.visible = true;
        }
    }
}

```

4. LevelUI.cs (Control Visual de Paneles y Canvas)

```

using UnityEngine;

public class LevelUI : MonoBehaviour
{
    [Header("Paneles de UI")]
    [SerializeField] private GameObject languagePanel;
    [SerializeField] private GameObject pausePanel;
    [SerializeField] private GameObject optionsPanel;
    [SerializeField] private GameObject gameOverPanel;

    private void OnEnable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged += HandleStateUI;
        }
    }

    private void OnDisable()
    {
        if (LevelManager.Instance != null)

```

```

    {
        LevelManager.Instance.OnGameStateChanged -= HandleStateUI;
    }
}

private void HandleStateUI(GameState state)
{
    if (languagePanel) languagePanel.SetActive(state == GameState.LanguageSelect);
    if (pausePanel) pausePanel.SetActive(state == GameState.Paused);
    if (gameOverPanel) gameOverPanel.SetActive(state == GameState.GameOver);

    // Si se cambia de estado, cerramos el panel de opciones por defecto
    if (optionsPanel && state != GameState.Paused)
    {
        optionsPanel.SetActive(false);
    }
}

// Métodos públicos vinculables a los Botones de la UI (OnClick)
public void OnLanguageSelected()
{
    LevelManager.Instance.SetGameState(GameState.Playing);
}

public void OnResumeButtonPressed()
{
    LevelManager.Instance.SetGameState(GameState.Playing);
}

public void OnOpenOptionsPressed()
{
    if (pausePanel) pausePanel.SetActive(false);
    if (optionsPanel) optionsPanel.SetActive(true);
}

public void OnCloseOptionsPressed()
{
    if (optionsPanel) optionsPanel.SetActive(false);
    if (pausePanel) pausePanel.SetActive(true);
}
}

```

¿Por qué esta estructura cumple con la regla de subordinados?

- **Estructura fija en Inspector:** El objeto `LevelManager` mantiene sus 4 componentes dedicados, exactamente como se muestra en tu captura.
- **Añadir nuevos scripts (Subordinados) no altera a `LevelManager`:** Si mañana agregamos un `SpawnManager`, `AudioLevelController` o `ScoreManager`, estos scripts se suscriben a los eventos de `LevelManager` o leen sus propiedades públicas. **Nunca vas a tener que reabrir ni modificar `LevelManager.cs`** para hacer espacio a nuevos subordinados.

User prompt: "Audio level controller" puede formar parte del level manager, es algo de alto nivel. Lo demas si son subordinados.

Response: Tiene todo el sentido. El control general de sonido del nivel (música de fondo, pausar audios o ajustar volúmenes globales) es una responsabilidad de **alto nivel** que le pertenece al contenedor `LevelManager`.

Para no romper el principio de responsabilidad única (**S de SOLID**), agregamos un 5.º script dedicado en el objeto `LevelManager`: **`AudioLevelController.cs`**.

Este script coordinará la música y los efectos según el estado del juego que le informe `LevelManager`, sin acoplarse con la lógica de UI o de entrada.

Módulo de Alto Nivel: `AudioLevelController.cs`

Agregás este script como 5.º componente dentro del objeto vacío `LevelManager`:

```

using UnityEngine;

public class AudioLevelController : MonoBehaviour
{
    [Header("Fuentes de Audio del Nivel")]
    [SerializeField] private AudioSource bgmSource;
    [SerializeField] private AudioSource sfxSource;

    [Header("Clips de Música")]
    [SerializeField] private AudioClip levelBGM;
    [SerializeField] private AudioClip gameOverBGM;
}

```

```

[SerializeField] private AudioClip victoryBGM;

private void OnEnable()
{
    if (LevelManager.Instance != null)
    {
        LevelManager.Instance.OnGameStateChanged += HandleGameStateAudio;
    }
}

private void OnDisable()
{
    if (LevelManager.Instance != null)
    {
        LevelManager.Instance.OnGameStateChanged -= HandleGameStateAudio;
    }
}

private void HandleGameStateAudio(GameState state)
{
    switch (state)
    {
        case GameState.LanguageSelect:
            // Si querés una música suave de menú de inicio
            break;

        case GameState.Playing:
            if (bgmSource != null && !bgmSource.isPlaying)
            {
                bgmSource.clip = levelBGM;
                bgmSource.UnPause();
                if (!bgmSource.isPlaying) bgmSource.Play();
            }
            break;

        case GameState.Paused:
            if (bgmSource != null && bgmSource.isPlaying)
            {
                bgmSource.Pause(); // Pausa la música durante el menú de pausa
            }
            break;

        case GameState.GameOver:
            PlayBGM(gameOverBGM, loop: false);
            break;

        case GameState.Victory:
            PlayBGM(victoryBGM, loop: false);
            break;
    }
}

public void PlaySFX(AudioClip clip)
{
    if (sfxSource != null && clip != null)
    {
        sfxSource.PlayOneShot(clip);
    }
}

private void PlayBGM(AudioClip clip, bool loop = true)
{
    if (bgmSource == null || clip == null) return;

    bgmSource.Stop();
    bgmSource.clip = clip;
    bgmSource.loop = loop;
    bgmSource.Play();
}
}

```

Estructura Consolidada del Objeto `LevelManager`

Tu `GameObject` vacío **`LevelManager`** en la escena ahora cuenta con sus **5 scripts de alto nivel**:

1. **`LevelManager`**: Administrador de estado global (Puntaje, Vidas, Pausa/Juego).
2. **`LevelUI`**: Gestor de los paneles de UI (Selección de idioma, Pausa, Opciones, Game Over).

3. **MenuInputManager**: Captura de teclas de menú (Pausa / Cancelar vía `InputDataMap`).
4. **CursorController**: Control de visibilidad y bloqueo del cursor del mouse.
5. **AudioLevelController**: Control del BGM (música de fondo) y reproductor de SFX del nivel.

Criterio de subordinados mantenido: Cualquier nuevo script que agreguemos por debajo en la jerarquía (como el **SpawnManager** para enemigos e ítems que crearemos en la Fase 3, o los bloques/monedas) simplemente consumirá los eventos o métodos públicos de este grupo de alto nivel sin modificar sus scripts internos.

User prompt: ahora rompimos estos 2 scripts.

Response: La razón por la que se rompen es que esos scripts viejos intentaban **tomar el control del flujo por su cuenta** en lugar de ser subordinados que respondan a la autoridad de `LevelManager` y `LevelUI`.

En la nueva arquitectura:

1. **PauseMenu queda 100% absorbido** por `LevelUI.cs` y `MenuInputManager.cs` (ya no hace falta un script aparte para la pausa en el nivel).
2. **Main_Menu se adapta a la regla OCP**: solo maneja la navegación de escenas de alto nivel, suscribiéndose/notificando a `LevelManager` sin tocar variables privadas.

Aquí tienes los dos scripts refactorizados para que encajen limpios sin romper la arquitectura:

1. `Main_Menu.cs` (Para la Escena de Menú Principal)

```
using UnityEngine;
using UnityEngine.SceneManagement;

#if UNITY_EDITOR
using UnityEditor;
#endif

public class Main_Menu : MonoBehaviour
{
    [Header("Referencias de Paneles / Canvas")]
    [SerializeField] private GameObject mainMenuPanel;
    [SerializeField] private GameObject optionsPanel;

    private void Start()
    {
        // El menú principal siempre requiere cursor visible
        Cursor.lockState = CursorLockMode.None;
        Cursor.visible = true;
        Time.timeScale = 1f;

        ShowMainMenu();
    }

    public void LoadScene(string sceneName)
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene(sceneName);
    }

    public void ToggleOptions(bool showOptions)
    {
        if (optionsPanel) optionsPanel.SetActive(showOptions);
        if (mainMenuPanel) mainMenuPanel.SetActive(!showOptions);
    }

    public void ShowMainMenu()
    {
        ToggleOptions(false);
    }

    public void QuitGame()
    {
#if UNITY_EDITOR
        EditorApplication.isPlaying = false;
#else
        Application.Quit();
#endif
    }
}
```

2. `PauseMenu.cs` (Adaptado como Subordinado de `LevelUI`)

Si prefieres conservar el componente `PauseMenu` en el panel en lugar de unificarlo todo en `LevelUI`, el script debe funcionar como un **subordinado que reacciona a los eventos de `LevelManager`**, sin manipular `Time.timeScale` ni estados por su cuenta:

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class PauseMenu : MonoBehaviour
{
    [Header("Referencias UI")]
    [SerializeField] private GameObject pausePanel;

    private void OnEnable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged += HandleGameState;
        }
    }

    private void OnDisable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged -= HandleGameState;
        }
    }

    private void HandleGameState(GameState state)
    {
        if (pausePanel != null)
        {
            pausePanel.SetActive(state == GameState.Paused);
        }
    }

    // Métodos vinculables a los OnClick() de los botones del Panel de Pausa
    public void ResumeGame()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.SetGameState(GameState.Playing);
        }
    }

    public void ReturnToMainMenu(string mainMenuSceneName)
    {
        Time.timeScale = 1f;

        // Si existe una instancia persistente de GameManager, la destruimos al salir
        if (GameManager.Instance != null)
        {
            Destroy(GameManager.Instance.gameObject);
        }

        SceneManager.LoadScene(mainMenuSceneName);
    }
}
```

¿Por qué se soluciona la rotura?

- Incapacidad de sobrescribir estados:** Los scripts ya no modifican directamente propiedades como `currentState` ni invocan métodos inexistentes de `MenuInputManager` (como `ChangeLayer` o `SetInputBlocked`).
- `Time.timeScale` unificado:** El tiempo del juego se controla de manera centralizada en `LevelManager.SetGameState()`, evitando conflictos donde un script reanuda el tiempo pero la UI sigue en pausa.

User prompt: ya teníamos un script que tenía `GameState`. Este hay que cambiarlo.

Response: Tienes toda la razón. Ya contábamos con `GameEnums.cs`, por lo que debemos reutilizar el enum oficial `GaState` y sus enumeradores (`Init`, `WaveInPrep`, `WaveInProgress`, `WaveCompleted`, `Paused`, `GameOver`) para no duplicar ni generar conflictos de nombres en el proyecto.

A continuación actualizamos los scripts de alto nivel para que utilicen `GaState`:

1. `LevelManager.cs` (Usando `GaState`)

```

using System;
using UnityEngine;

public class LevelManager : MonoBehaviour
{
    public static LevelManager Instance { get; private set; }

    [Header("Configuración del Nivel")]
    [SerializeField] private int initialLives = 3;

    public int CurrentScore { get; private set; }
    public int CurrentLives { get; private set; }
    public GaState CurrentState { get; private set; }

    // Eventos expuestos para subordinados
    public event Action<GaState> OnGameStateChanged;
    public event Action<int> OnScoreChanged;
    public event Action<int> OnLivesChanged;

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    private void Start()
    {
        CurrentScore = 0;
        CurrentLives = initialLives;
        SetGameState(GaState.Init);
    }

    public void SetGameState(GaState newState)
    {
        CurrentState = newState;

        // Control centralizado del tiempo al pausar
        Time.timeScale = (newState == GaState.Paused) ? 0f : 1f;

        OnGameStateChanged?.Invoke(CurrentState);
    }

    public void TogglePause()
    {
        if (CurrentState == GaState.WaveInProgress || CurrentState == GaState.WaveInPrep || CurrentState == GaState.Paused)
        {
            SetGameState(GaState.Paused);
        }
        else if (CurrentState == GaState.Paused)
        {
            SetGameState(GaState.WaveInProgress);
        }
    }

    public void AddScore(int points)
    {
        CurrentScore += points;
        OnScoreChanged?.Invoke(CurrentScore);
    }

    public void ApplyDamage(int damage)
    {
        CurrentLives -= damage;
        OnLivesChanged?.Invoke(CurrentLives);

        if (CurrentLives <= 0)
        {
            SetGameState(GaState.GameOver);
        }
    }
}

```

2. LevelUI.cs (Actualizado con GaState)

```

using UnityEngine;

public class LevelUI : MonoBehaviour
{
    [Header("Paneles de UI")]
    [SerializeField] private GameObject languagePanel;
    [SerializeField] private GameObject pausePanel;
    [SerializeField] private GameObject optionsPanel;
    [SerializeField] private GameObject gameOverPanel;

    private void OnEnable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged += HandleStateUI;
        }
    }

    private void OnDisable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged -= HandleStateUI;
        }
    }

    private void HandleStateUI(GaState state)
    {
        if (languagePanel) languagePanel.SetActive(state == GaState.Init);
        if (pausePanel) pausePanel.SetActive(state == GaState.Paused);
        if (gameOverPanel) gameOverPanel.SetActive(state == GaState.GameOver);

        if (optionsPanel && state != GaState.Paused)
        {
            optionsPanel.SetActive(false);
        }
    }

    public void OnLanguageSelected()
    {
        LevelManager.Instance.SetGameState(GaState.WaveInProgress);
    }

    public void OnResumeButtonPressed()
    {
        LevelManager.Instance.SetGameState(GaState.WaveInProgress);
    }

    public void OnOpenOptionsPressed()
    {
        if (pausePanel) pausePanel.SetActive(false);
        if (optionsPanel) optionsPanel.SetActive(true);
    }

    public void OnCloseOptionsPressed()
    {
        if (optionsPanel) optionsPanel.SetActive(false);
        if (pausePanel) pausePanel.SetActive(true);
    }
}

```

3. CursorController.cs (Actualizado con GaState)

```

using UnityEngine;

public class CursorController : MonoBehaviour
{
    [SerializeField] private bool hideOnStart = true;

    private void OnEnable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged += HandleGameState;
        }
    }

    private void OnDisable()
    {

```

```

        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged -= HandleGameState;
        }
    }

    private void HandleGameState(GaState state)
    {
        bool isGameplay = (state == GaState.WaveInProgress || state == GaState.WaveInPrep);

        if (isGameplay && hideOnStart)
        {
            Cursor.lockState = CursorLockMode.Confined;
            Cursor.visible = false;
        }
        else
        {
            Cursor.lockState = CursorLockMode.None;
            Cursor.visible = true;
        }
    }
}

```

4. AudioLevelController.cs (Actualizado con GaState)

```

using UnityEngine;

public class AudioLevelController : MonoBehaviour
{
    [Header("Fuentes de Audio del Nivel")]
    [SerializeField] private AudioSource bgmSource;
    [SerializeField] private AudioSource sfxSource;

    [Header("Clips de Música")]
    [SerializeField] private AudioClip levelBGM;
    [SerializeField] private AudioClip gameOverBGM;

    private void OnEnable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged += HandleGameStateAudio;
        }
    }

    private void OnDisable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged -= HandleGameStateAudio;
        }
    }

    private void HandleGameStateAudio(GaState state)
    {
        switch (state)
        {
            case GaState.WaveInProgress:
            case GaState.WaveInPrep:
                if (bgmSource != null && !bgmSource.isPlaying)
                {
                    bgmSource.clip = levelBGM;
                    bgmSource.UnPause();
                    if (!bgmSource.isPlaying) bgmSource.Play();
                }
                break;

            case GaState.Paused:
                if (bgmSource != null && bgmSource.isPlaying)
                {
                    bgmSource.Pause();
                }
                break;

            case GaState.GameOver:
                PlayBGM(gameOverBGM, loop: false);
                break;
        }
    }
}

```

```

public void PlaySFX(AudioClip clip)
{
    if (sfxSource != null && clip != null)
    {
        sfxSource.PlayOneShot(clip);
    }
}

private void PlayBGM(AudioClip clip, bool loop = true)
{
    if (bgmSource == null || clip == null) return;

    bgmSource.Stop();
    bgmSource.clip = clip;
    bgmSource.loop = loop;
    bgmSource.Play();
}
}

```

5. PauseMenu.cs (Subordinado actualizado con GaState)

```

using UnityEngine;
using UnityEngine.SceneManagement;

public class PauseMenu : MonoBehaviour
{
    [Header("Referencias UI")]
    [SerializeField] private GameObject pausePanel;

    private void OnEnable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged += HandleGameState;
        }
    }

    private void OnDisable()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.OnGameStateChanged -= HandleGameState;
        }
    }

    private void HandleGameState(GaState state)
    {
        if (pausePanel != null)
        {
            pausePanel.SetActive(state == GaState.Paused);
        }
    }

    public void ResumeGame()
    {
        if (LevelManager.Instance != null)
        {
            LevelManager.Instance.SetGameState(GaState.WaveInProgress);
        }
    }

    public void ReturnToMainMenu(string mainMenuSceneName)
    {
        Time.timeScale = 1f;

        if (GameManager.Instance != null)
        {
            Destroy(GameManager.Instance.gameObject);
        }

        SceneManager.LoadScene(mainMenuSceneName);
    }
}

```

Con este ajuste, todo el proyecto queda perfectamente alineado al enum oficial **GaState** definido en `GameEnums.cs`.

User prompt: no me entendiste, GameEnums.cs hay que cambiarlo, no los otros. Ya estas rompiendo la regla que habiamos puesto de no adaptar los script del level manager a los subordinados.

Response: Tenés toda la razón, mala mía. Estaba rompiendo la regla que fijamos: los scripts de nivel superior (LevelManager, LevelUI, AudioLevelController, etc.) **no se tocan ni se adaptan a los de abajo**.

Para respetar la arquitectura, corregimos el enum en **GameEnums.cs** para que la capa base contenga la definición limpia que necesita nuestro LevelManager sin alterar los componentes de alto nivel.

GameEnums.cs Actualizado

Modificamos únicamente GameEnums.cs para reemplazar/consolidar GaState:

```
// GameEnums.cs
// Este script no necesita heredar de MonoBehaviour
```

```
public enum GaState
{
    LanguageSelect,
    Playing,
    Paused,
    GameOver,
    Victory
}

public enum EnemyID
{
    ZomBunny = 0,
    ZomBear = 1,
    Hellephant = 2
}

public enum CollectibleID
{
    Coin = 0,
    Health = 1,
    Ammo = 2,
    RosaShield = 3,
    ExtraLife = 4
}
```

De esta forma, la capa de alto nivel (LevelManager y sus 4 subordinados directos) queda **100% intacta** con los estados estándar (LanguageSelect, Playing, Paused, GameOver, Victory), y cualquier subordinado que se agregue de ahora en adelante simplemente consumirá los estados desde este GameEnums.cs.
